Selfie Pipes: Phased Implementation Plan

Oct 25, 2026 · @Creeplyn

Seven phases (0–6) take the course `selfie.c` from one process with no IPC to blocking pipes between processes. Each phase ends at a gate: `selfie.c` builds with gcc, compiles itself with zero warnings, still runs itself (selfie-on-selfie), and passes its own tests plus every earlier phase's tests. Every phase and every test below was built and run on a copy of your `selfie.c`, and the expected outputs are the ones those runs produced.

At a glance



implementation roadmap · 7 phases, 7 gates

Read it top to bottom: each phase uses what the one above it built, and the next phase starts only after its gate passes. Phase 5 is the core of the architecture; Phases 0–4 make it testable, and Phase 6 closes the last gap.

How to test every phase

A phase is done only when all four gate steps pass and every earlier phase's tests still give their expected output.

```
# 1. builds
gcc -w -O2 -m64 -D'uint64_t=unsigned long' selfie.c -o selfie
# 2. compiles itself: the output must contain no "error" or "warning" line
./selfie -c selfie.c
# 3. still runs itself (selfie-on-selfie); hello.c just writes "hello from
selfie"
./selfie -c selfie.c -m 64 -c hello.c -m 1
# 4. the phase's tests (show only the program's own output)
./selfie -c tests/p3_registry.c -m 2 | sed -n '/>>>>/,/<<<</p'
```

Pass rules:

- Step 2 passes when no line contains "error" or "warning". Step 3 passes when it prints "hello from selfie" and both terminating lines say exit code 0.
- A test passes when its output matches the expected output exactly: every line is `ok NN`, no `FAIL`, nothing missing. The `terminating … with exit code N` line must show the expected code.
- Regression: re-run every earlier phase's tests at each gate. The one planned exception is `p4_temporary.c`, which is valid only in Phase 4 and must deadlock from Phase 5 on.
- Tests are C* programs (selfie compiles them), so they use no `for`, `&&`, `||`, arrays or structs.
- A syscall's −1 reaches the program as 4294967295, because selfie shrinks syscall results to 32 bits (`sign_shrink(-1, SYSCALL_BITWIDTH)`). The tests compare against `MINUS1`.

Shared helpers. Paste them at the top of every test except `p0_write.c`, `p1_exit.c`, `p5_stream.c` and `p5_deadlock.c`:

```
uint64_t* out;
uint64_t MINUS1 = 4294967295; // -1 as seen by programs: syscall results are
32-bit

void put(uint64_t c) {
  *out = c;
  write(1, out, 1);
}

void check(uint64_t id, uint64_t pass) {
  if (pass) {
    put('o'); put('k');
  } else {
    put('F'); put('A'); put('I'); put('L');
  }
  put(' '); put('0' + id / 10); put('0' + id % 10); put(10);
}

void spin(uint64_t n) {
  // burn CPU so the timer interrupt hands the CPU to another process
  uint64_t i;
  i = 0;
  while (i < n)
    i = i + 1;
}
```

Every test that uses `check` sets `out = malloc(8);` first in `main`.

Phase 0 — Trustworthy console output

Outcome: `write(1, buf, n)` prints exactly `n` bytes for every `n`.

Goal: every later test reports through `write()`, so fix the existing bug first. Without this fix a correct pipe looks broken: the prototype printed "byte byts" until it was fixed.

Requirements

1. **R0.1** In `implement_write` (line 7831), after `copy_buffer` succeeds, terminate again exactly after `size` bytes: `store_character((char*) IO_buffer, size, 0);`. Reason: `copy_buffer` copies whole 8-byte words and overwrites the terminator written before it.
2. **R0.2** Nothing else changes: same return value, same −1 error path.

3. **R0.3** The gate passes (build, self-compile, selfie-on-selfie).

Testing protocol

1. Before changing anything, run `p0_write.c` on the unmodified `selfie.c`. Expected: `abcdefgh|0123456789|XY` (the bug, reproduced).
2. Apply R0.1, then run gate steps 1–3.
3. Run `p0_write.c` again. Expected: `abc|0123456789|X`, exit code 0.

```
// tests/p0_write.c
uint64_t main() {
  write(1, "abcdefghij", 3);
  write(1, "|", 1);
  write(1, "0123456789", 10);
  write(1, "|", 1);
  write(1, "XY", 1);
  write(1, "\n", 1);
  return 0;
}
```

Passes when: step 1 shows the bug, which proves the test can detect it; step 3 matches exactly (R0.1, R0.2); the gate passes (R0.3).

Phase 1 — Context status and a scheduler

Outcome: `mipster()` can manage any number of contexts. It never resumes a BLOCKED or EXITED context, and it returns only when every context has exited. With one context it behaves exactly as before.

Goal: blocking needs another process to switch to. Build the scheduler first, while there is still only one process and any regression is easy to spot.

Requirements

1. **R1.1** `CONTEXTENTRIES` goes from 32 to 34 (line 2026). Field 32 = status, field 33 = next waiter. Add both rows to the layout comment. Add the constants `CTX_READY = 0`, `CTX_BLOCKED = 1` and `CTX_EXITED = 2`.
2. **R1.2** Add `get_status` / `set_status` and `get_next_waiter` / `set_next_waiter` next to the other context accessors (after line 2138).

3. **R1.3** At the end of `init_context` (line ~11305), set status to READY and next waiter to 0. This also covers contexts reused from `free_contexts`, which `smalloc` does not zero.
4. **R1.4** `schedule(from)`: round robin over `used_contexts`, starting after `from` and wrapping to the head. It returns the first READY context, considers `from` itself last, and returns 0 when no context is READY.
5. **R1.5** In `mipster()` (line 11996):
   a. Save `get_exception(from_context)` before calling `handle_exception`, which resets it.
   b. On `EXIT`, mark the context EXITED instead of returning.
   c. Switch contexts on a timer interrupt or when the context is no longer READY; otherwise keep running the same context.
   d. When `schedule()` returns 0, return the exit code of the context that exited last.
   e. Leave the "dispatch exception handling to parent" branch as it is.
6. **R1.6** With one context, everything visible stays the same: program output and exit codes.

Testing protocol

1. Run gate steps 1–3. Step 3 is the strongest test here: selfie-on-selfie takes hundreds of timer interrupts, and every one of them now goes through `schedule()` with a single context.
2. Run `p1_exit.c`. Expected: `survived timer interrupts`, exit code 7.
3. Regression: `p0_write.c`.

```
// tests/p1_exit.c
uint64_t main() {
  uint64_t i;
  i = 0;
  // long enough for many timer interrupts with a single context
  while (i < 2000000)
    i = i + 1;
  write(1, "survived timer interrupts\n", 26);
  return 7;
}
```

Passes when: the gate passes and selfie-on-selfie still terminates with exit code 0 (R1.1–R1.5); `p1_exit.c` prints its line and ends with exit code 7, which proves the exit code still comes through (R1.5.4, R1.6). The rule that the machine keeps running until every context has exited can only be tested once a second process exists. `p2_exit.c` in Phase 2 covers it.

Phase 2 — fork(): a second process

Outcome: `fork()` creates a second process with a private copy of code, data, heap and stack. The child gets 0, the parent gets a positive PID, and the Phase 1 scheduler runs both.

Goal: a pipe connects two processes, but `selfie_run()` only ever creates one. Without `fork()` there is nobody to wake a blocked reader.

Requirements

1. **R2.1** Add `SYSCALL_FORK = 220` to the syscall ID block (line 1309) and a prototype `void emit_fork();`.
2. **R2.2** `emit_fork()`: a wrapper with no arguments, following the `emit_count_syscalls` pattern (`a7 = SYSCALL_FORK`, `ecall`, `jalr`). Call it from the bootstrapping list (line 6299).
3. **R2.3** Add `syscall_return(context, value)`: it sets `a0 = value` and advances the PC by `INSTRUCTIONSIZE`. Every pipe syscall reuses it later.
4. **R2.4** `implement_fork`:
   a. Call `create_context(MY_CONTEXT, 0)`.
   b. Copy the name, the code/data/heap segment bounds and the program break before mapping any page, because `map_page` sorts pages into the low or high range using the program break.
   c. Copy all 32 registers.
5. **R2.5** Copy every mapped page in `[lowest_lo, highest_lo)` and `[lowest_hi, highest_hi)`: `palloc()` a new frame, copy `PAGEFRAMESIZE` bytes word by word, then `map_page` it into the child. No frame is shared.
6. **R2.6** The child resumes at PC + 4 with `a0 = 0`. The parent gets a positive, increasing PID through `syscall_return`.
7. **R2.7** Add a branch to `handle_system_call`: `a7 == SYSCALL_FORK` → `implement_fork(context)`.

Testing protocol

1. Run gate steps 1–3.
2. Run `p2_fork.c`. Expected, in this order: `ok 04`, `ok 05`, `ok 01`, `ok 02`, `ok 03`, `ok 06`, `ok 07`, `ok 08`, then exit code 0. Here is why: syscalls don't switch processes, so the parent prints 04–05 first. Its `spin` triggers a timer interrupt, the child runs 01–03 and exits, then the parent resumes with 06–08.
3. Run `p2_exit.c`. Expected: `ok 01`, `ok 02`, exit code 5. The parent exits first; the machine must keep running the child and report the exit code of the context that exited last.
4. Regression: `p0_write.c`, `p1_exit.c`.

```
// tests/p2_fork.c  (helpers pasted above)
uint64_t x;

uint64_t main() {
  uint64_t pid;
  uint64_t local;
  uint64_t* heap;

  out = malloc(8);
  heap = malloc(8);

  x = 1;
  local = 42;
  *heap = 7;

  pid = fork();

  if (pid == 0) {
    // child: sees copies of everything, then changes its copies
    check(1, x == 1);
    check(2, local == 42);
    check(3, *heap == 7);
    x = 2;
    local = 43;
    *heap = 9;
    exit(0);
  }

  check(4, pid > 0);
  check(5, pid != MINUS1);
```

```
  // let the child run (timer interrupt), then confirm our copies are
untouched
  spin(400000);

  check(6, x == 1);
  check(7, local == 42);
  check(8, *heap == 7);

  return 0;
}
```

```
// tests/p2_exit.c  (helpers pasted above)
uint64_t main() {
  out = malloc(8);

  if (fork() == 0) {
    // child outlives the parent
    spin(400000);
    check(2, 1);
    exit(5);
  }

  check(1, 1);
  return 0;
}
```

Passes when:

- `ok 01` – `ok 03`: the child saw copied data, stack and heap (R2.4, R2.5).
- `ok 04` – `ok 05`: the parent got a valid PID (R2.6).
- `ok 06` – `ok 08`: the child's writes did not reach the parent's memory (R2.5, isolation).
- The line order matches: a timer interrupt switched processes (R1.5).
- `p2_exit.c` ends with exit code 5 (R1.5.2, R1.5.4).
- The gate and the regression tests pass.

Phase 3 — Pipe objects and the registry

Outcome: `pipe_create()` returns the lowest free ID (1–15), and `pipe_destroy(id)` frees it. Both return −1 for an invalid ID or a full registry.

Goal: get the data structures and the ID lookup right on their own, before any byte moves through them.

Requirements

1. **R3.1** Add `SYSCALL_PIPE_CREATE = 500`, `SYSCALL_PIPE_DESTROY = 503` and their prototypes. Add `emit_pipe_create()` (no arguments) and `emit_pipe_destroy()` (one argument: `a0` = ID), both called from the bootstrapping list.
2. **R3.2** The pipe object is 9 words from `zmalloc`: id, buffer, r, w, n, readers head/tail, writers head/tail. Document it with a layout comment like the machine context's, and add `get_` / `set_` accessors for id, buffer, r, w and n. The ring storage is `zmalloc(round_up(PIPE_CAPACITY, 8))`, with `PIPE_CAPACITY = 16` bytes so blocking is easy to trigger later.
3. **R3.3** The registry `pipe_table` is allocated with `zmalloc(MAX_PIPES * 8)` on the first create. `MAX_PIPES = 16`, and ID 0 is never valid.
4. **R3.4** `lookup_pipe(id)` returns 0 if the table doesn't exist yet, if `id == 0`, if `id >= MAX_PIPES` (which also rejects −1), or if the slot is empty.
5. **R3.5** `implement_pipe_create` takes the lowest free slot and returns its ID. When the registry is full it returns −1.
6. **R3.6** `implement_pipe_destroy` returns −1 for an invalid ID. Otherwise it clears the slot and returns 0. Waking sleepers comes in Phase 6. The storage is not freed because selfie has no free list, so leave a TODO.
7. **R3.7** Put the kernel code just before `handle_system_call` (line ~11857), where `map_page`, `palloc` and `create_context` are already defined. Add both dispatch branches.

Testing protocol

1. Run gate steps 1–3.
2. Run `p3_registry.c`. Expected: `ok 01` to `ok 08`, exit code 0.
3. Regression: the Phase 0–2 tests.

```
// tests/p3_registry.c  (helpers pasted above)
uint64_t main() {
  uint64_t p1;
  uint64_t p2;
  uint64_t i;

  out = malloc(8);
```

```
  p1 = pipe_create();
  p2 = pipe_create();
  check(1, p1 == 1);
  check(2, p2 == 2);
  check(3, pipe_destroy(p1) == 0);
  check(4, pipe_destroy(p1) == MINUS1);
  check(5, pipe_create() == 1);
  check(6, pipe_destroy(0) == MINUS1);
  check(7, pipe_destroy(99) == MINUS1);

  // IDs 3 .. 15 are still free
  i = 0;
  while (i < 13) {
    pipe_create();
    i = i + 1;
  }
  check(8, pipe_create() == MINUS1);

  return 0;
}
```

Passes when:

- `ok 01` – `ok 02`: lowest free ID (R3.3, R3.5).
- `ok 03` – `ok 04`: destroy works, and a second destroy is rejected (R3.6).
- `ok 05`: a freed ID is reused (R3.5).
- `ok 06` – `ok 07`: out-of-range IDs are rejected (R3.4).
- `ok 08`: a full registry returns −1 (R3.5).
- The gate passes (R3.1, R3.7) and the regression tests pass.

Phase 4 — Moving bytes without blocking

Outcome: `pipe_write` and `pipe_read` move `min(requested, available)` bytes in FIFO order, byte-exact, from and to buffers at any alignment. For now, when nothing can move they return 0.

Goal: get the data path and memory safety right in a single process, where every failure is deterministic and easy to trace, before adding concurrency.

Requirements

1. **R4.1** Add `SYSCALL_PIPE_WRITE = 501` and `SYSCALL_PIPE_READ = 502`, with 3-argument wrappers (`a0` = ID, `a1` = buffer, `a2` = size) that follow `emit_read`. Add them to the bootstrapping list and to the dispatch.
2. **R4.2** `guest_word(context, vaddr)` returns the physical address of the word that holds any given byte, or 0 if that byte isn't accessible:
   a. Round `vaddr` down to its word and check it with `is_virtual_address_valid` and `is_data_stack_heap_address`.
   b. If the page has never been touched, map a fresh `palloc()` frame, as `handle_page_fault` does.
   c. Do not use `copy_buffer` for pipe data. It requires 8-byte-aligned addresses and copies whole words, so a 5-byte read would overwrite 3 bytes past the buffer.
3. **R4.3** Move data one byte at a time with `load_character` / `store_character`. Write: `buffer[w] = byte; w = (w + 1) % C; n = n + 1`. Read is the mirror image. Stop when `size` bytes have moved or the pipe is full (write) or empty (read).
4. **R4.4** Return values:
   a. `k`, the number of bytes moved. Short transfers are allowed, and callers loop.
   b. 0 when `size == 0`.
   c. −1 for an invalid ID.
   d. −1 for a bad address before any byte moved; the pipe stays unchanged.
   e. `k` for a bad address after some bytes moved.
5. **R4.5** Temporary rule: when nothing can move (a read of an empty pipe, a write to a full one), return 0 right away. Phase 5 replaces exactly these two branches.
6. **R4.6** Check the ID and `size == 0` before checking for empty/full, so those calls can never block once Phase 5 exists.

Testing protocol

1. Run gate steps 1–3.
2. Run `p4_transfer.c`. Expected: `ok 01` to `ok 33`, exit code 0.
3. Run `p4_temporary.c`. Expected: `ok 01` to `ok 03`, exit code 0.
4. Regression: the Phase 0–3 tests.

```
// tests/p4_transfer.c  (helpers pasted above)
// single process; never blocks, so it stays valid after Phase 5
uint64_t main() {
  uint64_t p;
```

```
  uint64_t* buf;
  uint64_t* big;
  uint64_t msg;

  out = malloc(8);
  buf = malloc(8);

  p = pipe_create();

  check(1, pipe_write(p, "ABCDEFGHIJ", 10) == 10);
  *buf = 0;
  check(2, pipe_read(p, buf, 4) == 4);
  check(3, *buf == 'A' + 'B' * 256 + 'C' * 65536 + 'D' * 16777216);
  *buf = 0;
  check(4, pipe_read(p, buf, 100) == 6);
  check(5, *buf == 'E' + 'F' * 256 + 'G' * 65536 + 'H' * 16777216 + 'I' *
4294967296 + 'J' * 1099511627776);

  // short write: only 16 bytes fit
  msg = (uint64_t) "abcdefghijklmnopqrstuvwxyz0123456789ABCD";
  check(6, pipe_write(p, (uint64_t*) msg, 40) == 16);
  check(7, pipe_read(p, buf, 8) == 8);
  check(8, *buf == *((uint64_t*) msg));
  check(9, pipe_read(p, buf, 8) == 8);
  check(10, *buf == *((uint64_t*) msg + 1));

  // unaligned source
  check(11, pipe_write(p, (uint64_t*) (msg + 3), 2) == 2);
  *buf = 0;
  check(12, pipe_read(p, buf, 2) == 2);
  check(13, *buf == 'd' + 'e' * 256);

  // a 1-byte read must not touch the other bytes of the word
  check(14, pipe_write(p, "Q", 1) == 1);
  *buf = 'x' + 'y' * 256 + 'z' * 65536;
  check(15, pipe_read(p, buf, 1) == 1);
  check(16, *buf == 'Q' + 'y' * 256 + 'z' * 65536);

  // unaligned destination
  check(17, pipe_write(p, "RS", 2) == 2);
  *buf = 0;
  check(18, pipe_read(p, (uint64_t*) ((uint64_t) buf + 5), 2) == 2);
  check(19, *buf == 'R' * 1099511627776 + 'S' * 281474976710656);
```

```
  // destination on a heap page the program never touched
  big = malloc(16384);
  check(20, pipe_write(p, "Z", 1) == 1);
  check(21, pipe_read(p, big + 1024, 1) == 1);
  check(22, *(big + 1024) == 'Z');

  // bad address: error, and the byte stays in the pipe
  check(23, pipe_write(p, (uint64_t*) 8, 1) == MINUS1);
  check(24, pipe_write(p, "W", 1) == 1);
  check(25, pipe_read(p, (uint64_t*) 8, 1) == MINUS1);
  *buf = 0;
  check(26, pipe_read(p, buf, 1) == 1);
  check(27, *buf == 'W');

  // zero sizes and bad IDs
  check(28, pipe_write(p, buf, 0) == 0);
  check(29, pipe_read(p, buf, 0) == 0);
  check(30, pipe_write(99, buf, 1) == MINUS1);
  check(31, pipe_read(99, buf, 1) == MINUS1);

  check(32, pipe_destroy(p) == 0);
  check(33, pipe_write(p, buf, 1) == MINUS1);

  return 0;
}
```

```
// tests/p4_temporary.c  (helpers pasted above)
// Phase 4 only: empty/full return 0 instead of blocking
uint64_t main() {
  uint64_t p;
  uint64_t* buf;

  out = malloc(8);
  buf = malloc(8);
  p = pipe_create();

  check(1, pipe_read(p, buf, 1) == 0);
  check(2, pipe_write(p, "abcdefghijklmnopqrstuvwxyz", 26) == 16);
  check(3, pipe_write(p, "a", 1) == 0);

  return 0;
}
```

Passes when:

Checks	What they prove	Requirements
`ok 01` – `ok 05`	FIFO order, exact bytes, short read	R4.3, R4.4
`ok 06` – `ok 10`	a short write stops at the 16-byte capacity	R4.3, R4.4
`ok 11` – `ok 13`	unaligned source buffer	R4.2
`ok 14` – `ok 16`	a read leaves the rest of the word alone	R4.2, R4.3
`ok 17` – `ok 19`	unaligned destination buffer	R4.2
`ok 20` – `ok 22`	a never-touched heap page is mapped on demand	R4.2
`ok 23` – `ok 27`	bad address → −1, no byte lost	R4.4
`ok 28` – `ok 31`	size 0 and bad IDs	R4.4, R4.6
`ok 32` – `ok 33`	a destroyed ID can't be used	R3.6
`p4_temporary` `ok 01` – `ok 03`	empty read and full write return 0 for now	R4.5

The gate (R4.1) and the regression tests must pass too.

Phase 5 — Blocking, wakeups and deadlock detection

Outcome: a read of an empty pipe or a write to a full one puts the process to sleep instead of returning 0. The next transfer in the other direction wakes it, and it retries. If every remaining process is asleep, selfie reports a deadlock (exit code 28) instead of hanging.

Goal: this is the core of the pipe architecture: a producer and a consumer that wait for each other without spinning.

Requirements

1. **R5.1** Queue offsets `PIPE_READERS = 5` and `PIPE_WRITERS = 7`: the head is at `pipe + q` and the tail at `pipe + q + 1`. `enqueue_waiter` appends the context in FIFO order through its next-waiter field and marks it BLOCKED. No queue nodes are allocated.
2. **R5.2** `wake_all_waiters(pipe, q)` sets every context in the queue to READY, clears its next-waiter field and empties the queue. Waking all of them is safe because each one rechecks.
3. **R5.3** Replace the two R4.5 branches: enqueue the context and return without advancing the PC and without touching `a0`. When the context runs again, its `ecall` executes again with `a0` – `a2` and `a7` unchanged, so the recheck comes for free.
4. **R5.4** After a write that moved at least one byte, wake the readers. After a read that moved at least one byte, wake the writers.
5. **R5.5** No lock is needed: the `implement_*` handlers run to completion inside the emulator, and the timer only fires between guest instructions. Checking and enqueuing is therefore already one atomic step. Say this in your report.
6. **R5.6** Deadlock detection: add `EXITCODE_DEADLOCK = 28` and `number_of_blocked_contexts()`. When `schedule()` returns 0 while some context is BLOCKED, print `deadlock: N context(s) blocked and none ready` and return 28.
7. **R5.7** Add a `debug_pipes` flag (default 0) that prints every block and wake. The protocol uses it in step 4.

Testing protocol

1. Run gate steps 1–3.
2. Run `p5_stream.c`, which sends 58 bytes through the 16-byte pipe, so both sides block. Expected: `Pipes carry bytes from one process to another, in order.` and exit code 0.
3. Run `p5_retry.c`. Expected: `r2=A` then `r1=B`, exit code 0. The rule behind it: each reader prints exactly once, one gets A and the other gets B.

4. Set `debug_pipes = 1`, rebuild, and run `p5_retry.c` again. Expected trace (addresses will differ): both readers block, both wake, `r2=A`, the other reader blocks again, wakes again, `r1=B`.
5. Run `p5_deadlock.c`. Expected: `deadlock: 1 context(s) blocked and none ready`, exit code 28.
6. Planned change: `p4_temporary.c` now prints nothing and ends with the same deadlock message and exit code 28. That proves the R4.5 branches are gone.
7. Regression: `p0_write`, `p1_exit`, `p2_fork`, `p2_exit`, `p3_registry`, `p4_transfer`.

```
// tests/p5_stream.c  (no helpers needed)
uint64_t main() {
  uint64_t p;
  uint64_t sent;
  uint64_t got;
  uint64_t r;
  uint64_t msg;
  uint64_t len;
  uint64_t* buf;

  msg = (uint64_t) "Pipes carry bytes from one process to another, in
order.\n";
  len = 58;

  p = pipe_create();

  if (fork() == 0) {
    // child = writer: short writes are allowed, so loop until everything is
sent
    sent = 0;
    while (sent < len) {
      r = pipe_write(p, (uint64_t*) (msg + sent), len - sent);
      sent = sent + r;
    }
    exit(0);
  }

  // parent = reader, 5 bytes per call
  buf = malloc(8);
  got = 0;
  while (got < len) {
    r = pipe_read(p, buf, 5);
```

```
    write(1, buf, r);
    got = got + r;
  }

  pipe_destroy(p);

  return 0;
}
```

```
// tests/p5_retry.c  (helpers pasted above)
// two readers sleep on one pipe; each byte wakes both, only one gets it, the
other sleeps again
uint64_t p;

void reader(uint64_t id) {
  uint64_t* buf;

  buf = malloc(8);
  *buf = 0;

  pipe_read(p, buf, 1);

  put('r'); put('0' + id); put('='); put(*buf); put(10);

  exit(0);
}

uint64_t main() {
  out = malloc(8);

  p = pipe_create();

  if (fork() == 0)
    reader(1);
  if (fork() == 0)
    reader(2);

  spin(400000);
  pipe_write(p, "A", 1);
  spin(400000);
  pipe_write(p, "B", 1);
```

```
  spin(400000);

  return 0;
}
```

```
// tests/p5_deadlock.c  (no helpers needed)
// a reader with no writer: the scheduler must report a deadlock
uint64_t main() {
  uint64_t p;
  uint64_t* buf;

  p = pipe_create();
  buf = malloc(8);
  pipe_read(p, buf, 1);

  return 0;
}
```

Expected `debug_pipes` trace for step 4:

```
./selfie: context 0x...8BD0 blocks on pipe 1
./selfie: context 0x...4990 blocks on pipe 1
./selfie: pipe 1 wakes context 0x...8BD0
./selfie: pipe 1 wakes context 0x...4990
r2=A
./selfie: context 0x...4990 blocks on pipe 1
./selfie: pipe 1 wakes context 0x...4990
r1=B
```

Passes when:

- `p5_stream` prints the message exactly: blocking in both directions plus retries (R5.1–R5.4).
- In the trace, a reader that loses the race goes back to sleep: wake-all followed by a recheck (R5.2, R5.3).
- `p5_deadlock` and `p4_temporary` end with exit code 28 (R5.6).
- The gate and the regression tests pass.

One side effect to mention in your report: each retried `ecall` counts as another syscall in `count_syscalls()`.

Phase 6 — Destroy wakes sleepers, and the final regression

Outcome: destroying a pipe wakes every process sleeping on it. Each one retries, finds the ID invalid and gets −1. The full test suite passes, including pipes running inside selfie-on-selfie.

Goal: a destroyed pipe must not leave anyone asleep forever. Then confirm the whole architecture end to end.

Requirements

1. **R6.1** In `implement_pipe_destroy`, clear the registry slot first, then call `wake_all_waiters` for the readers and the writers.
2. **R6.2** Add no special case to read or write. The retried `ecall` reaches the invalid-ID path from R4.4 and returns −1.
3. **R6.3** Every test from Phases 0–5 still gives its expected output, and pipes work when selfie runs itself.

Testing protocol

1. Negative check, before applying R6.1, on the Phase 5 build:
   a. `p6_destroy_reader.c` must print `ok 01`, `ok 03`, then the deadlock message, exit code 28.
   b. `p6_destroy_writer.c` must print `ok 01`, `ok 02`, then the deadlock message, exit code 28.
   c. This proves both tests catch a missing wakeup.
2. Apply R6.1 and run gate steps 1–3.
3. `p6_destroy_reader.c`: `ok 01`, `ok 02`, `ok 03`, exit code 0. `p6_destroy_writer.c`: `ok 01`, `ok 02`, `ok 03`, exit code 0.
4. Full regression: every test from Phases 0–5. `p4_temporary.c` still deadlocks, as planned.
5. Pipes inside selfie-on-selfie:
   a. `./selfie -c selfie.c -m 64 -c tests/p5_stream.c -m 2` prints the message, and both terminating lines say exit code 0.
   b. The same command with `tests/p4_transfer.c` prints 33 `ok` lines.

```
// tests/p6_destroy_reader.c  (helpers pasted above)
// a reader blocked on an empty pipe gets -1 when the pipe is destroyed
uint64_t main() {
  uint64_t p;
```

```
  uint64_t* buf;

  out = malloc(8);
  buf = malloc(8);
  p = pipe_create();

  if (fork() == 0) {
    check(2, pipe_read(p, buf, 4) == MINUS1);
    exit(0);
  }

  spin(400000);
  check(1, pipe_destroy(p) == 0);
  spin(400000);
  check(3, pipe_destroy(p) == MINUS1);

  return 0;
}
```

```
// tests/p6_destroy_writer.c  (helpers pasted above)
// a writer blocked on a full pipe gets -1 when the pipe is destroyed
uint64_t main() {
  uint64_t p;

  out = malloc(8);
  p = pipe_create();

  if (fork() == 0) {
    check(1, pipe_write(p, "abcdefghijklmnopqrstuvwxyz", 26) == 16);
    check(3, pipe_write(p, "a", 1) == MINUS1);
    exit(0);
  }

  spin(400000);
  check(2, pipe_destroy(p) == 0);
  spin(400000);

  return 0;
}
```

Passes when: the negative check fails as described; both destroy tests print `ok 01` to `ok 03` (R6.1, R6.2); and steps 4–5 all match (R6.3).

Final sign-off:

- [ ] Gate steps 1–3 pass on the final `selfie.c`
- [ ] `p0_write` → `abc|0123456789|X`
- [ ] `p1_exit` → message, exit code 7
- [ ] `p2_fork` → `ok 04 05 01 02 03 06 07 08`; `p2_exit` → exit code 5
- [ ] `p3_registry` → `ok 01` – `ok 08`
- [ ] `p4_transfer` → `ok 01` – `ok 33`; `p4_temporary` → deadlock, exit code 28
- [ ] `p5_stream` → the message; `p5_retry` → `r2=A`, `r1=B`; `p5_deadlock` → exit code 28
- [ ] `p6_destroy_reader` and `p6_destroy_writer` → `ok 01` – `ok 03`
- [ ] Pipes inside selfie-on-selfie (step 5)